Field Hours

CSCE 668 Distributed Algorithms and Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Message passing, memory, and faults

You name the model before you name the algorithm. In the message-passing model, n processes exchange messages and hold no common variables. In the shared-memory model they read and write registers. A synchronous round lets every process send, receive, and update once. An asynchronous schedule may delay any message by any finite amount. With n = 5 and at most 1 crash, a proof written for one model does not transfer to the other.

You also name the failure type, because the same five processes behave differently under it. A crash stops a process and leaves its last messages intact. A Byzantine process can send different contents to different neighbors. Fairness of the scheduler is a separate knob: you may assume every non-crashed process takes infinitely many steps, or you may allow a process to be postponed for 100 steps. Write those three choices at the top of the argument.

Write these choices before the protocol.

Message passing or shared memory

Synchronous or asynchronous steps

Crash or Byzantine faults

5 processes, at most 1 crash

Five processes, one crash

1. You set n = 5, message passing, asynchronous delivery, and at most 1 crash.

2. A proof that assumes every message arrives in the same round does not apply, because the schedule may delay a message.

3. A proof that allows forged Byzantine sends also does not apply. A crash does not invent messages. The model line is part of the claim.

Problem 1. You have 5 processes and at most 1 crash, but the proof you found is about messages. Does it transfer to shared memory?

Answer. No. Shared memory uses registers, not in-flight messages. Matching n and the crash bound is not enough. You redo the argument in the memory model.

Problem 2. A process stops, and every message it already sent stays as sent. What failure is that, and how many does the small model allow?

Answer. A crash, not a Byzantine fault. The small model allows at most 1 crash among 5 processes.

Lecture 2 · 90 minutes

Leader election on a ring

You elect a leader on a unidirectional ring of n processes with distinct numeric identifiers. Each process sends its identifier clockwise. A process that sees a smaller identifier swallows it. A process that sees a larger identifier forwards it. The largest identifier travels all the way around the ring and becomes the leader. In the worst order of identifiers the message count is n(n+1)/2. For n = 4 that count is 10.

You can see those 10 messages when identifiers 1, 2, 3, and 4 are placed in the worst order. Identifier 4 travels 4 hops and returns to its owner, which is how that owner learns it is the leader. The smaller identifiers still contribute 1 + 2 + 3 hops before a larger identifier swallows them. No process has a global view. The ring edges and the numeric identifiers are the whole mechanism.

Election on a directed ring.

1. Send your identifier

2. Swallow a smaller one

3. Forward a larger one

4. Largest id returns home

Worst-case messages for n = 4

1. Identifiers are 1, 2, 3, and 4. The worst-case message total is 4*5/2 = 10.

2. Identifier 4 travels 4 hops and returns. Its owner then knows it is the leader.

3. The other identifiers contribute 1 + 2 + 3 = 6 hops before they are swallowed. Added to the 4 hops, the total is 10.

Problem 1. A unidirectional ring has n = 6 processes with distinct identifiers. How many messages does the worst case send?

Answer. 6*7/2 = 21.

Problem 2. Identifier 9 is the largest on a ring of 4. How many hops does it travel before its owner knows?

Answer. It travels once around the ring, which is 4 hops, and the returning copy tells the owner it is the leader.

Lecture 3 · 90 minutes

Mutual exclusion for two processes

You implement mutual exclusion for 2 processes with Peterson's three shared variables: a flag for each process and one turn variable. A process raises its flag, sets turn to the other process, and waits while the other flag is raised and the turn still names the other process. Then it enters the critical section and lowers its flag on exit. The turn variable breaks the tie when both flags are raised. At most one process is inside.

You check progress and bounded waiting on the same 2-process protocol. If both processes are waiting, the

process not named by turn stops waiting, so one of them enters. After a process sets turn to the other, that other process enters at most once before the turn moves back. Those are the progress and waiting claims. Exclusion, progress, and waiting are three separate sentences, all carried by the same three registers.

Peterson's protocol for two processes.

Shared variables

Two flags and one turn

Wait while the other is favored

Lower your flag on exit

Requirements

At most one process inside

Someone enters if both wait

Overtake at most once

Both processes try at once

1. Process 0 and process 1 both raise flags. Process 0 writes turn = 1, then process 1 writes turn = 0.

2. Process 0 waits only while turn names process 1. Turn is 0, so process 0 enters even though both flags are up.

3. Process 1 waits while flag 0 is up and turn is 0. Exclusion holds. After process 0 lowers its flag, process 1 can enter.

Problem 1. How many shared variables does this 2-process protocol use, and what are they?

Answer. Three: one flag for each process and one turn variable.

Problem 2. Both flags are raised and turn names process 1. Which process may enter?

Answer. Process 1 may enter. Process 0 waits, because its wait condition is that the other flag is raised and turn names process 1.

Lecture 4 · 90 minutes

Logical clocks and happens-before

You order events with a logical clock when physical time is unavailable. Each process keeps an integer, starting at 0. A local step sets the clock to its current value plus 1. A send carries the sender clock. On receipt the receiver sets its clock to 1 plus the maximum of its own clock and the value in the message. If event a happens before event b, the clock of a is strictly smaller. The converse can fail.

You can watch the rule on two processes and three events. Process P takes a step at clock 1, a second step at clock 2, and sends 2. Process Q is already at clock 5. On receipt Q sets its clock to max(5, 2) + 1, which equals 6. The clocks along this happens-before chain are 1, then 2, then 6. Equal clocks on different processes do not, by themselves, order those two events.

Lamport clock update on one message.

Local step adds 1 -> Send the clock value -> Receiver takes the max -> Then add 1

Clocks 1, 2, and 6

1. Process P steps from 0 to 1, then from 1 to 2, and sends the value 2.

2. Process Q is already at 5. On receipt it computes max(5, 2) + 1 = 6.

3. The chain of clocks is 1, then 2, then 6. Each event happens before the next. An equal clock on another process would not force an order.

Problem 1. A receiver's clock is 4 and the message carries 4. What clock does the receive event get?

Answer. max(4, 4) + 1 = 5.

Problem 2. Event a has clock 2, event b has clock 9, and a happens before b. Is that allowed, and does a larger clock prove happens-before?

Answer. It is allowed, because happens-before forces a strictly smaller clock. The converse fails: a larger clock does not by itself prove happens-before.

Lecture 5 · 90 minutes

Consistent snapshots

You record a global state with a snapshot that does not freeze the processes. Each process records its local state once and sends a marker on every outgoing channel. Channels are FIFO. If a process receives a marker on a channel and has not recorded yet, it records immediately, then sends its own markers. The recorded state plus the in-flight messages form the snapshot. For 3 processes each process emits 2 markers if the network is complete.

You accept the snapshot only when the cut is consistent. If the cut contains a receive, it must contain the matching send. On a FIFO channel the marker separates those messages: anything sent before the marker belongs in the channel state, and anything sent after it does not. A cut that holds 1 receive and omits its send is illegal. A cut that holds the send and not yet the receive is allowed.

Marker rule on one FIFO channel.

1. Record local state

2. Send a marker

3. Record the channel state

4. Reject an illegal cut

One data message still in flight

1. Process P records its state and sends a marker. One earlier data message has not yet arrived at Q.

2. Q receives the marker, records its local state, and records that data message as the channel contents.

3. The cut contains the send and not the receive, which is allowed. A cut with the receive and without

the send would be rejected.

Problem 1. A cut contains 1 receive and omits the matching send. Is the snapshot consistent?

Answer. No. If the receive is in the cut, the matching send must be in the cut.

Problem 2. How many markers does one process send in a complete network of 3 processes?

Answer. Each process has degree 2, so it sends 2 markers, one on each outgoing channel.

Lecture 6 · 90 minutes

Consensus under crash faults

You state consensus with three properties and one failure bound. Agreement says every non-faulty process that decides picks the same value. Validity says that if every process proposes the same bit, the decision is that bit. Termination says every non-faulty process decides. In a synchronous network with at most f crash faults, f + 1 rounds suffice. For f = 1 you run 2 rounds, because after a crash-free round the remaining views have met.

You do not keep that round count when the network is asynchronous. With even 1 possible crash, no deterministic protocol gives agreement, validity, and termination on every schedule. The obstruction is a bivalent configuration: a state from which both decisions are still reachable. An adversary keeps the protocol inside bivalent states by delaying one process. Randomization or partial synchrony is how you leave that impossibility. Deterministic async consensus is not available.

Crash consensus in two timing models.

Synchronous, f = 1

Decide in 2 rounds

Flood the proposals

Agreement and validity

Asynchronous, 1 crash

Deterministic protocols fail

Bivalent states remain

Delay one process

Why 2 rounds when f = 1

1. In round 1 every process sends its proposal. One process may crash in that round, so some of its messages never arrive.

2. Round 2 repeats the exchange. The bound f = 1 forbids a second crash, so this round lets the surviving views meet.

3. Every non-faulty process then decides the same value. If every proposal was the bit 1, validity forces the decision 1.

Problem 1. At most f = 3 crash faults in a synchronous network. How many rounds suffice?

Answer. f + 1 = 4 rounds.

Problem 2. Why can a deterministic asynchronous protocol not guarantee all three consensus properties if 1 crash is possible?

Answer. An adversary can hold the execution in a bivalent state by delaying one process, so the protocol cannot decide without risking disagreement or an invalid bit.

Lecture 7 · 90 minutes

Byzantine counts and quorums

You count processes before you promise Byzantine agreement. With oral messages and at most f traitors, you need at least 3f + 1 processes. For f = 1 that number is 4. With only 3 processes a traitor can show one honest process a command to attack and the other a command to retreat, and the honest processes cannot tell who lied. Crash faults are cheaper: a majority is enough to pick a value already proposed, because two majorities intersect.

You store that intersection with quorums in shared memory. For n = 5 a majority quorum has size 3, because floor(5/2) + 1 = 3. Any two quorums share at least one process, since 3 + 3 - 5 = 1. A writer deposits a value and a timestamp at one quorum. A reader queries a quorum and keeps the largest timestamp. That intersection of 1 is why the reader cannot miss the latest completed write.

Headcount depends on the fault you named.

Byzantine f = 1 needs 4

Three processes can be fooled

Majority of 5 is 3

Two quorums share 1 process

A write and a read on 5 processes

1. A writer stores a value with timestamp 8 at a quorum of 3 processes.

2. A reader contacts another quorum of 3. The two sets intersect in at least 3 + 3 - 5 = 1 process.

3. That shared process still holds timestamp 8, so the reader returns the value stamped 8 rather than an older write.

Problem 1. At most f = 2 Byzantine traitors, oral messages. What is the smallest n the bound allows?

Answer. 3*2 + 1 = 7.

Problem 2. n = 5. What is the majority quorum size, and what is the least intersection of two such quorums?

Answer. The quorum size is floor(5/2) + 1 = 3. Two quorums intersect in at least 3 + 3 - 5 = 1 process.

Lecture 8 · 90 minutes

What you can now check on a small system

You can now match a problem to a model and finish a small proof. On a ring of 4 processes you can count leader-election messages up to the worst case of 10. For 2 processes you can check exclusion, progress, and bounded waiting on three shared variables. You can assign Lamport clocks, reject an inconsistent cut, and separate the synchronous crash bound of f + 1 rounds from the asynchronous impossibility with 1 crash.

You can also move between failures and memory. A Byzantine bound of 4 processes for 1 traitor is a different budget from a crash quorum of 3 out of 5. The skill is to write the assumptions in one line, run the mechanism on a tiny system, and say which property holds: a unique leader, a consistent snapshot, or a decision that honest processes share. Larger systems use the same invariants with a bigger n.

From the model line to a property you can check.

1. State n and the faults

2. Run the small mechanism

3. Check the invariant

4. Name the property

Clocks witness a bad snapshot

1. A send has Lamport clock 4 and its receive has clock 7, so the send happens before the receive.

2. A proposed cut includes the receive and omits the send.

3. The snapshot rule forbids that cut. The clocks 4 and 7 witness that the missing send is ordered before the included receive.

Problem 1. Combine logical clocks with snapshot consistency. A send is stamped 4 and its receive is stamped 7. The cut includes the receive and omits the send. Why is the cut illegal?

Answer. The clocks 4 then 7 witness happens-before along the message, so the send is ordered before the receive. A consistent cut that contains the receive must also contain that send. This cut fails the rule.

Problem 2. Combine the synchronous crash bound with the Byzantine headcount. For f = 1, how many synchronous crash rounds suffice, and how many processes do oral-message Byzantine agreement need?

Answer. Crash faults in synchrony take f + 1 = 2 rounds. Byzantine oral messages need 3*1 + 1 = 4 processes. The round count is not a substitute for the headcount.